

Одна схема

```
Response r = given()
    .spec(BASE_SPEC)
    .when().get("/booking");

assertThat(r.statusCode()).isEqualTo(200);
BookingResponse dto = r.as(BookingResponse.class);
```

REST Assured — HTTP, Jackson — JSON↔DTO, AssertJ — проверки, JUnit — запуск.

JSON-массив сразу в DTO

```
List<BookingIdResponse> items =
    r.as(new TypeRef<>() {});

assertThat(items).isNotEmpty()
    .allSatisfy(x ->
        assertThat(x.bookingid()).isPositive());
```

TypeRef сохраняет generic type. GPath для списка не нужен.

Plain response

```
assertThat(r.statusCode()).isEqualTo(201);
assertThat(r.contentType()).startsWith("text/plain");
assertThat(r.asString()).isEqualTo("Created");
```

Что учим, а что нет

|            |                                                        |
|------------|--------------------------------------------------------|
| Основа     | Response, DTO, TypeRef, AssertJ                        |
| Точечно    | path/query params, Map для PATCH                       |
| Не маршрут | Большие body/matcher цепочки, schema, сложный JsonPath |

## POST DTO

```
Response r = given().spec(BASE_SPEC)
    .contentType(ContentType.JSON)
    .body(expected)
    .post("/booking");
CreateBookingResponse actual =
    r.as(CreateBookingResponse.class);
```

## Wrapper DTO

```
record CreateBookingResponse(
    int bookingid,
    BookingResponse booking) {}
```

Java-модель повторяет структуру JSON.

## Read-back

```
Response read = given().spec(BASE_SPEC)
    .pathParam("id", actual.bookingid())
    .get("/booking/{id}");
BookingResponse saved = read.as(BookingResponse.class);
assertThat(saved).usingRecursiveComparison()
    .isEqualTo(expected);
```

Mutation response не доказывает сохранение.

## Query filter

```
.queryParams("firstname", expected.firstname())
.queryParam("lastname", expected.lastname())
.get("/booking")
```

Ответ снова читаем как `List<BookingIdResponse>`.

### Request specification

```
new RequestSpecBuilder()
    .setBaseUri(BASE_URL)
    .setAccept(ContentType.JSON)
    .addFilter(new ErrorLoggingFilter())
    .build();
```

URL, safe headers, config и failure logging — в спец.  
Body и expected — в тесте.

### Negative response

```
Response r = given().spec(BASE_SPEC)
    .pathParam("id", unknownId)
    .log().ifValidationFails()
    .get("/booking/{id}");
r.then().log().ifValidationFails();
assertThat(r.statusCode()).isEqualTo(404);
```

### Диагностика

1. Method, URL и params.
2. Безопасные headers/body.
3. Status, content type, response body.
4. Свой ID и expected DTO.
5. Какой шаг flow упал.

## Token как DTO

```
Response r = given().spec(BASE_SPEC)
    .contentType(ContentType.JSON)
    .body(new AuthRequest("admin", "password123"))
    .post("/auth");
AuthResponse auth = r.as(AuthResponse.class);
assertThat(auth.token()).isNotBlank();
```

## Parameterized negative

```
@ParameterizedTest
@CsvSource({"admin, bad", "bad, password123"})
void invalid(String user, String pass) {
    AuthErrorResponse error = response.as(...);
    assertThat(error.reason())
        .isEqualTo("Bad credentials");
}
```

## Использование

```
given().spec(BASE_SPEC)
    .cookie("token", auth.token())
    .pathParam("id", id)
    .body(updateDto)
    .put("/booking/{id}");
```

### Общая формула

before → authorized action → exact status → GET → saved DTO → changed + preserved.

### PUT

```
.cookie("token", token)
.contentType(JSON).body(updated)
.put("/booking/{id}");
BookingResponse saved = get(id).as(...);
assertThat(saved).usingRecursiveComparison()
    .isEqualTo(updated);
```

### PATCH

```
Map<String, Object> patch =
    Map.of("lastname", changed);
// PATCH, затем GET
softly.assertThat(saved.lastname()).isEqualTo(changed);
softly.assertThat(saved.firstname())
    .isEqualTo(original.firstname());
```

### Защита и delete

PUT без token → 403, затем GET должен вернуть original. DELETE → 201, затем GET → 404.

Финальный flow

- 1. Unique request DTO.
- 2. POST → response DTO + ID.
- 3. Зарегистрировать cleanup.
- 4. GET → recursive comparison.
- 5. PATCH → GET changed/preserved.
- 6. DELETE → GET 404.

Независимость

- Нет static mutable ID.
- Каждый тест создаёт своё.
- Expected локален.
- Cleanup работает после падения.
- Тест проходит отдельно и в suite.

Архитектура после повторений

|          |                                     |
|----------|-------------------------------------|
| tests    | scenario + assertions               |
| client   | endpoint calls, возвращает Response |
| models   | request/response DTO                |
| fixtures | unique valid data                   |
| specs    | общая HTTP config                   |

Критерий готовности

Самостоятельно построить request, DTO, assertions, read-back, negative state check, cleanup и понять причину падения.